

Dockerfiles & Kubernetes Manifests Explained

All the code explained here is in the GitHub repository:
github.com/nitin1094/three-tier-app-eks

A Beginner's Line-by-Line Walkthrough — every Dockerfile in `Application-Code/` and every manifest in `Kubernetes-Manifests-file/`, explained one line at a time.

What this guide covers

Backend Dockerfile	Six lines that package the Node.js API — and why their order matters
Frontend Dockerfile	The React image, the one line that differs, and the production version
.dockerignore	The invisible file that keeps builds fast and images correct
Database manifests	Secret, PersistentVolume, PersistentVolumeClaim, Deployment & Service
Backend manifests	Rolling updates, private image pulls, and liveness vs readiness probes
Frontend manifests	Why the React tier uses a public URL while the API uses an internal one
The ALB Ingress	One public address, routed by path to two Services
Appendices	Field reference, placeholders, error decoder, interview Q&A & glossary

Written in plain language for readers with only basic Docker and Kubernetes knowledge. Every file is quoted *verbatim* from the repository — nothing simplified, nothing invented — so you can open the real code alongside this guide and follow along. A green “From the repo:” link under each chapter heading points at the exact file.

Table of Contents

Every entry is a live link — click it to jump straight to that topic.

How the Pieces Fit	4
1. What This Guide Explains	4
2. Docker in Five Minutes	5
3. Kubernetes in Five Minutes	6
The Dockerfiles (Application-Code/)	8
4. The Backend Dockerfile, Line by Line	8
5. The Frontend Dockerfile, Line by Line	11
6. The .dockerignore Files	12
7. From Dockerfile to Running Image	14
The Manifests (Kubernetes-Manifests-file/)	15
8. How to Read These Manifests	15
9. Database - secrets.yaml	16
10. Database - pv.yaml and pvc.yaml	18
11. Database - deployment.yaml	20
12. Database - service.yaml	25
13. Backend - deployment.yaml	27
14. Backend - service.yaml	31
15. Frontend - deployment.yaml	31
16. Frontend - service.yaml	33
17. The Ingress - ingress.yaml	34
18. Putting It All Together	36
Appendices	38
19. Field Reference Cheat Sheet	38
20. Placeholders You Must Replace	40

21. Common Errors and What They Mean	41
22. Interview Questions and Answers	41
23. Glossary	43
24. Links and Repository	44

How the Pieces Fit

What this guide covers, the app behind the files, and just enough Docker and Kubernetes to make every line that follows meaningful.

1. What This Guide Explains

This is a **code-reading guide**, not a deployment tutorial. Its whole job is to open every Dockerfile and every Kubernetes YAML file in the three-tier project and explain them **line by line**, in plain English, so that nothing in them is a mystery to you.

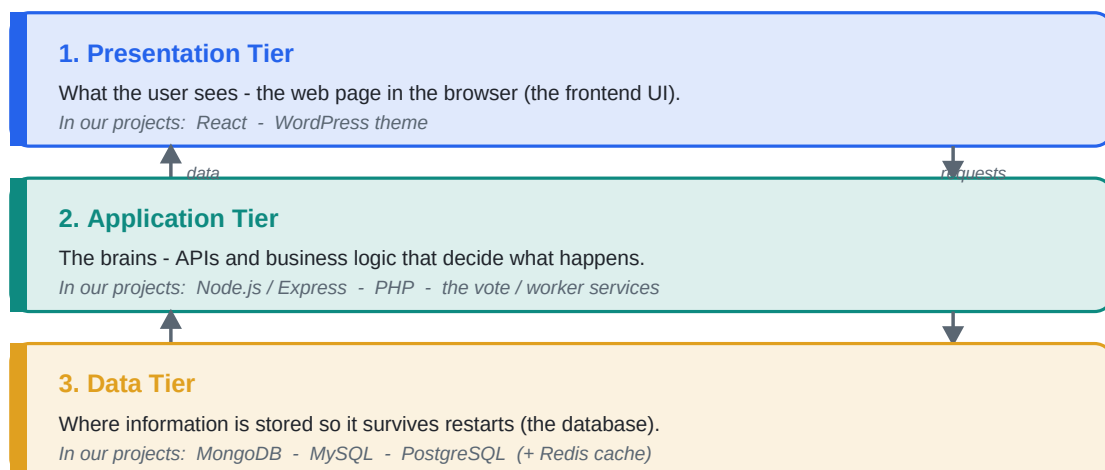
You need only basic Docker and Kubernetes knowledge to follow along: you should know roughly what a container is and that Kubernetes runs containers for you. Everything beyond that is explained here as it comes up.

The project these files belong to

The app is a small **to-do list** split into three tiers:

Tier	What it is	Technology	Listens on
Presentation	The web page you click on	React (Create React App)	port 3000
Application	The API that handles the to-dos	Node.js + Express + Mongoose	port 3500
Data	Where the to-dos are actually stored	MongoDB 4.4.6	port 27017

The three tiers of a modern web application



Where the original code lives

Every file quoted in this guide is copied **verbatim** from the project's GitHub repository. Nothing has been rewritten or simplified — if a line looks odd, that is genuinely how it is in the repo, and this guide explains why.

From the repo: github.com/nitin1094/three-tier-app-eks

Throughout the guide, a green **"From the repo:"** line under a heading links straight to the exact file being discussed, so you can open it side by side while you read.

The two folders we will read

```

three-tier-app-eks/
+-- Application-Code/  # the app + its BUILD instructions
| +-- frontend/
| | +-- Dockerfile  # Part II, chapter 5
| | +-- .dockerignore  # Part II, chapter 6
| +-- backend/
| +-- Dockerfile  # Part II, chapter 4
| +-- .dockerignore  # Part II, chapter 6
+-- Kubernetes-Manifests-file/  # the RUN instructions for the cluster
    +-- Database/
    | +-- secrets.yaml  # Part III, chapter 9
    | +-- pv.yaml  # Part III, chapter 10
    | +-- pvc.yaml  # Part III, chapter 10
    | +-- deployment.yaml  # Part III, chapter 11
    | +-- service.yaml  # Part III, chapter 12
    +-- Backend/
    | +-- deployment.yaml  # Part III, chapter 13
    | +-- service.yaml  # Part III, chapter 14
    +-- Frontend/
    | +-- deployment.yaml  # Part III, chapter 15
    | +-- service.yaml  # Part III, chapter 16
    +-- ingress.yaml  # Part III, chapter 17

```

KEY IDEA A **Dockerfile** is a *build-time* file — it describes how to package one tier into an image. A **Kubernetes manifest** is a *run-time* file — it describes how that image should behave once it is running in a cluster. Two different jobs, two different languages, two different folders.

2. Docker in Five Minutes

Before we read the Dockerfiles, here is just enough Docker to make every line meaningful.

The three levels: Dockerfile, image, container

Think of cooking:

- A **Dockerfile** is the **recipe** — a plain text file of instructions, written once.
- An **image** is the **frozen meal** made by following that recipe — a read-only, shippable bundle containing the app, its libraries, and the operating-system files it needs.
- A **container** is the **meal being eaten** — a running copy of the image. You can start many containers from one image, and they do not interfere with each other.

You turn a recipe into a frozen meal with `docker build`, and a frozen meal into a served meal with `docker run` (or, in our case, by letting Kubernetes do the running).

ANALOGY An image is like a class in programming and a container is like an object created from it. The class is written once; you can make as many objects as you like from it.

Images are built in layers

Every instruction in a Dockerfile (`FROM`, `COPY`, `RUN`, ...) creates one **layer** — a saved snapshot of the filesystem at that point. Layers stack on top of each other to form the final image.

This matters for one very practical reason: **Docker caches layers**. When you rebuild, Docker walks your Dockerfile from the top and reuses each cached layer until it reaches an instruction whose inputs have changed. From that instruction down, everything is rebuilt.

That single fact explains the shape of both Dockerfiles in this project, as you will see in chapter 4.

The instructions used in this project

Instruction	What it does
FROM	Picks the starting image to build on top of. Always the first real instruction.
WORKDIR	Sets the "current folder" inside the image for every instruction after it. Creates it if missing.
COPY	Copies files from your machine (the <i>build context</i>) into the image.
RUN	Executes a command while building , and saves the result as a new layer.
CMD	The default command to run when a container starts . Runs once, at run time — not at build time.

NOTE `RUN` happens during `docker build`; `CMD` happens during `docker run`. Mixing these two up is the single most common beginner confusion with Dockerfiles. A handy test: if it should already be *baked into* the image, it is a `RUN`. If it should happen *fresh each time* the container starts, it is `CMD`.

3. Kubernetes in Five Minutes

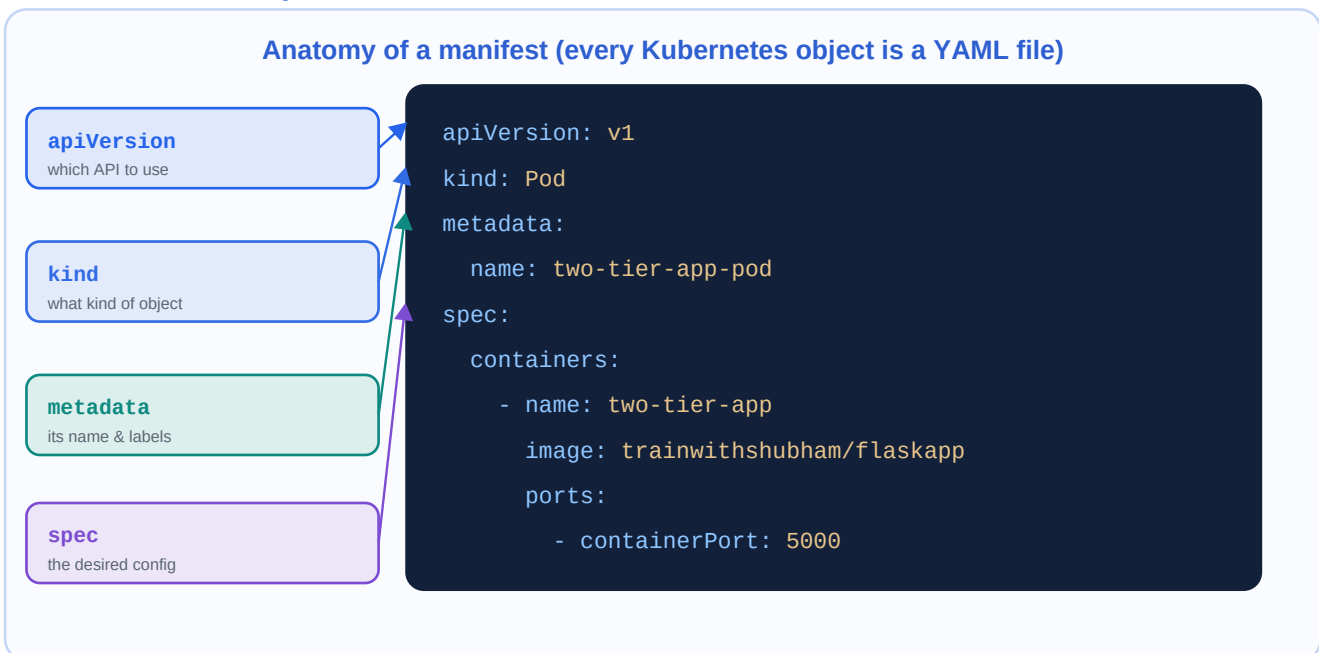
Now the other half. Here is the vocabulary you need to read the manifests in Part III.

Everything is a described object

You never tell Kubernetes *how* to do something. You hand it a YAML file describing **what you want**, and Kubernetes continuously works to make reality match that description. This is called being **declarative**, and the loop that keeps reality in sync is called **reconciliation**.

So when you read a manifest, read it as a wish, not as a script: *"I want one MongoDB pod, using this image, with this volume attached."*

The four fields every manifest starts with



Every single YAML file in `Kubernetes-Manifests-file/` opens with the same four keys:

Field	Meaning	Example from this project
<code>apiVersion</code>	Which version of the Kubernetes API this object belongs to	<code>apps/v1</code> , <code>v1</code> , <code>networking.k8s.io/v1</code>

Field	Meaning	Example from this project
<code>kind</code>	What type of object you are creating	<code>Deployment</code> , <code>Service</code> , <code>Secret</code> , <code>Ingress</code>
<code>metadata</code>	Its name, its namespace, and its labels	<code>name: frontend</code> , <code>namespace: three-tier</code>
<code>spec</code>	The actual wish — everything specific to that kind	<code>replicas</code> , <code>image</code> , <code>ports</code> , <code>volumes</code> ...

Once you recognise this skeleton, every manifest becomes far less intimidating: the top four lines tell you *what* you are looking at, and `spec` is the only part that really differs between files.

TIP `apiVersion: v1` is used for the oldest, most fundamental objects (`Service`, `Secret`, `PersistentVolume`, `PersistentVolumeClaim`). `apps/v1` is for workload controllers (`Deployment`). `networking.k8s.io/v1` is for networking objects (`Ingress`). If you get an *"no matches for kind"* error, a wrong `apiVersion` is usually the cause.

The object types you will meet

Object	In one sentence
Pod	The smallest unit Kubernetes runs — one or more containers that share a network address. You rarely write one by hand.
Deployment	A manager that keeps N identical pods alive and rolls out new versions safely. All three tiers here are Deployments.
Service	A stable name and IP in front of a changing set of pods. Pods come and go; the Service name never changes.
Secret	A place to keep credentials out of your Deployment files, injected into containers as environment variables.
PersistentVolume (PV)	A piece of storage that exists independently of any pod — so data survives a pod restart.
PersistentVolume Claim (PVC)	A pod's <i>request</i> for storage: "I need 1 GiB." Kubernetes matches it to a PV.
Ingress	An HTTP router at the edge of the cluster — one public address, routed by URL path to different Services.
Namespace	A folder inside the cluster that groups related objects. Everything here lives in <code>three-tier</code> .

Labels and selectors — the glue

This is the one concept that ties almost every manifest in this project together, so it is worth getting right early.

A **label** is just a key/value sticker you put on an object:

```
labels:
  role: frontend  # a sticker saying "this pod is the frontend"
```

A **selector** is a search for that sticker:

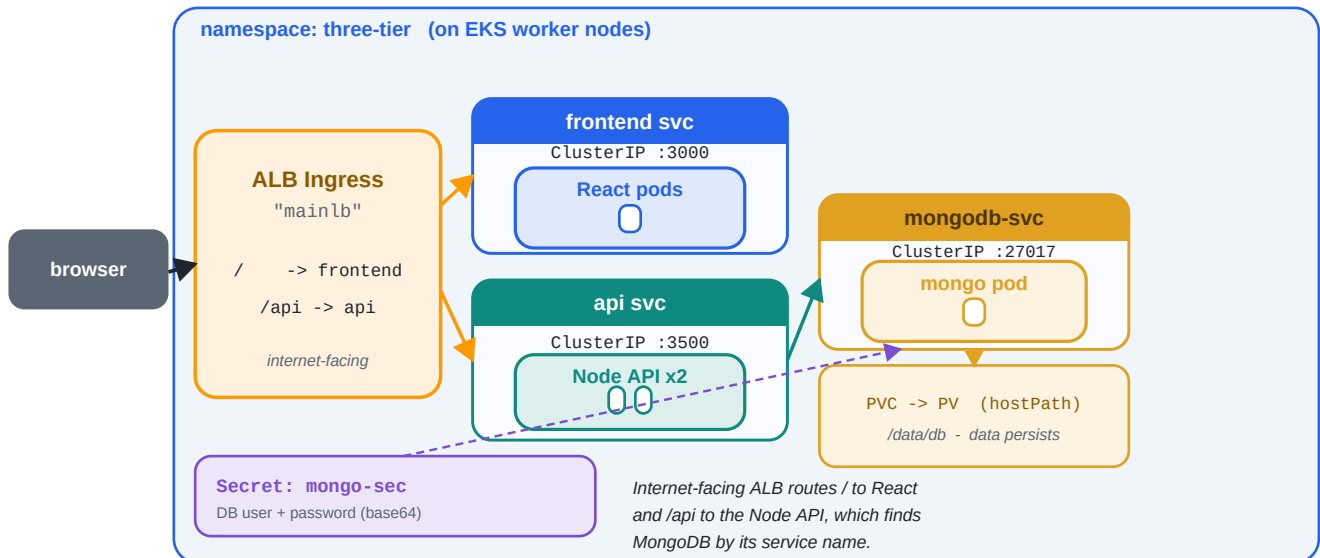
```
selector:
  role: frontend  # "find me everything wearing the 'role: frontend' sticker"
```

Deployments use a selector to know which pods they own. Services use a selector to know which pods to send traffic to. Nothing is wired by IP address — **everything is wired by matching labels**.

WATCH OUT If a Service's selector does not exactly match the labels on the pods, the Service will have **no endpoints** and traffic will silently go nowhere. There is no error message. When something "just doesn't connect" in Kubernetes, mismatched labels are the first thing to check with `kubectl get endpoints -n three-tier`.

How this project's objects fit together

The three-tier app on Amazon EKS



Reading that picture from the outside in:

- 1 A user hits one public address — the **ALB** created by the Ingress.
- 2 The Ingress routes `/` to the **frontend** Service and `/api` to the **api** Service.
- 3 Each Service forwards to the pods its selector matches.
- 4 The api pods talk to MongoDB by the Service name `mongodb-svc` — no IP addresses anywhere.
- 5 MongoDB writes its files to a **PersistentVolume** so the to-dos survive a restart, and reads its username and password from a **Secret**.

Everything in Part III is one of those five sentences, written out in YAML.

PART II

The Dockerfiles (Application-Code/)

The backend and frontend Dockerfiles line by line, the `.dockerignore` that makes them work, and how the images reach Amazon ECR.

4. The Backend Dockerfile, Line by Line

From the repo: [Application-Code/backend/Dockerfile](#)

We start with the backend because it is the simpler of the two. Here is the whole file — six lines, nothing hidden:

```
FROM node:14
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["node", "index.js"]
```

That is the entire recipe for the API tier. Let us take it one line at a time.

Line 1 — FROM node:14

```
FROM node:14 # start from the official Node.js 14 image on Docker Hub
```

Every image is built **on top of another image**. FROM picks that starting point, called the **base image**.

`node:14` means: the official image published by the Node.js team, tagged `14`. That image already contains a Debian Linux filesystem, the Node.js 14 runtime, and `npm`. Because all of that is already there, you do not have to install Linux or Node yourself — you inherit them.

The part after the colon is the **tag**. `node:14` is a *floating* tag: it points at the newest 14.x release at the moment you build. `node:14.21.3` would be an exact, frozen version.

WATCH OUT Node 14 reached **end of life in April 2023** — it no longer receives security patches. The project uses it because that is what the original tutorial used, and because the app's dependencies (React 17, Mongoose 5) were written for that era. For anything real, move to a supported release such as `node:20-alpine`, and pin the exact version so your builds are repeatable.

TIP Notice there is no `-alpine` or `-slim` suffix here. Plain `node:14` is the *full* Debian image — around 900 MB. `node:14-alpine` is the same Node on a minimal Alpine Linux base and is roughly 10 times smaller. Smaller images pull faster onto your cluster nodes and have a smaller security surface.

Line 2 — WORKDIR /usr/src/app

```
WORKDIR /usr/src/app # cd into this folder inside the image; create it if missing
```

`WORKDIR` is the Docker equivalent of `cd`, with one bonus: if the folder does not exist, Docker creates it for you.

From this line onwards, every `COPY`, `RUN` and `CMD` in the file is relative to `/usr/src/app`. That is why the next line can say `COPY package*.json ./` — the `./` means `/usr/src/app/`.

`/usr/src/app` is simply a long-standing convention for "where the application source lives" in Node images. Nothing breaks if you choose `/app` instead; it just has to be consistent.

NOTE Always use `WORKDIR` rather than `RUN cd /some/dir`. A `RUN cd` only affects that one instruction — the next instruction starts back where it was, because each instruction runs in its own fresh shell. `WORKDIR` is persistent for the rest of the build *and* for the running container.

Lines 3 and 4 — copy the dependency list, then install it

```
COPY package*.json ./ # copy ONLY package.json and package-lock.json
RUN npm install # download the libraries listed in those files
```

This pair of lines is the most important idea in the whole file, and it is deliberately written this way.

`package*.json` is a wildcard that matches both `package.json` (the list of dependencies) and `package-lock.json` (the exact versions that were last installed). Those two small files are copied in **on their own**, before any application code.

`RUN npm install` then reads them and downloads every dependency into a `node_modules` folder inside the image. For the backend that is `express`, `mongoose`, and `cors`, plus everything *they* depend on — hundreds of packages, and the slowest step in the build.

Why not just copy everything at once and then install? **Because of layer caching.**

Remember from chapter 2 that Docker reuses a cached layer until an instruction's inputs change. Here is what that buys you:

You changed...	<code>COPY package*.json</code>	<code>RUN npm install</code>	<code>COPY . .</code>
Only <code>index.js</code> (typical)	cache hit — unchanged	cache hit — skipped entirely	rebuilt (fast)
<code>package.json</code> (added a library)	cache miss	re-runs (slow, but necessary)	rebuilt

Because you edit application code far more often than you add libraries, this ordering turns a two-minute rebuild into a two-second one. If the file instead said `COPY . .` followed by `RUN npm install`, **every single code change** would invalidate the copy layer and force a full re-install.

KEY IDEA Order your Dockerfile from **least likely to change** at the top to **most likely to change** at the bottom. That one rule is what makes rebuilds fast, and it is why dependency files are always copied and installed before source code.

TIP In a production pipeline, prefer `RUN npm ci --omit=dev` over `npm install`. `npm ci` installs *exactly* what `package-lock.json` says (reproducible, and it fails loudly if the lock file is out of sync), and `--omit=dev` leaves out test-only packages you do not ship.

Line 5 — `COPY . .`

```
COPY . . # copy the rest of the app source into /usr/src/app
```

Now the actual code goes in: `index.js`, `db.js`, `routes/tasks.js`, `models/task.js`.

The first `.` is the source — the **build context**, meaning the folder you pointed `docker build` at. The second `.` is the destination, which thanks to `WORKDIR` means `/usr/src/app`.

You might worry that this would also copy the `node_modules` folder from your laptop, overwriting the one `npm install` just created inside the image. It would — except that `.dockerignore` excludes it. That file is chapter 6, and this is exactly why it matters.

Line 6 — `CMD ["node", "index.js"]`

```
CMD ["node", "index.js"] # what to run when a container starts
```

`CMD` sets the **default command** for containers created from this image. Nothing runs during the build — this is a note left for later, executed when the container starts.

`node index.js` starts the Express server. Looking at the app's own `index.js`, that server:

```
const port = process.env.PORT || 3500; // default port 3500
app.listen(port, () => console.log(`Listening on port ${port}...`));
```

...which is why every Kubernetes manifest in Part III refers to port **3500** for the API.

The square brackets are the **exec form**, and they are the form you want. Written this way, `node` becomes the container's main process (PID 1) and receives shutdown signals directly, so Kubernetes can stop the pod cleanly. The alternative *shell form*, `CMD node index.js`, wraps the command in `/bin/sh -c`, which swallows those signals and makes pods take ten extra seconds to die.

NOTE `CMD` is a *default*, not a lock. Anything you pass after the image name in `docker run` replaces it, and in Kubernetes the `command:` and `args:` fields of a container do the same — which is precisely what the MongoDB manifest does in chapter 11.

The backend Dockerfile in one picture

Line	Instruction	Runs when?	Creates a layer?	Why it is here
1	<code>FROM node:14</code>	build	yes (the base)	Gives you Linux + Node 14 for free
2	<code>WORKDIR /usr/src/app</code>	build	tiny	Sets the working folder for everything below
3	<code>COPY package*.json ./</code>	build	yes	Dependency list only — changes rarely
4	<code>RUN npm install</code>	build	yes (the big one)	Downloads express, mongoose, cors
5	<code>COPY . .</code>	build	yes	The app source — changes constantly
6	<code>CMD ["node", "index.js"]</code>	run	no	Starts the API on port 3500

5. The Frontend Dockerfile, Line by Line

From the repo: [Application-Code/frontend/Dockerfile](#)

Now the React side:

```
FROM node:14
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
CMD [ "npm", "start" ]
```

If that looks familiar, it should — **five of the six lines are identical to the backend**. The reason is simple: both tiers are Node projects, so both are packaged the same way. Everything explained in chapter 4 applies here unchanged.

Only the last line differs, and the difference is worth understanding properly.

The one line that changes — `CMD [ "npm", "start" ]`

```
CMD [ "npm", "start" ] # run the "start" script from package.json
```

Instead of calling `node` directly, this runs the `start` script defined in the frontend's `package.json`:

```
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build"
}
```

So the container actually runs `react-scripts start` — the **Create React App development server**. That server compiles the React code in memory, serves it on port `3000`, and watches for file changes to hot-reload the browser.

WATCH OUT This is a **development server running in production**, and it is the single biggest compromise in this project. `react-scripts start` is slow to boot, uses far more memory than needed, serves unminified code, and prints a warning telling you not to use it this way. It works for a learning project, but you would not ship it to real users.

What a production frontend image looks like instead

The proper pattern is a **multi-stage build**: compile the React app in one stage, then copy only the finished static files into a tiny web-server image. For reference, that would look like this (this is *not* in the repo — it is the upgrade path):

```
# ----- stage 1: build the static files -----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci # reproducible install from package-lock.json
COPY . .
RUN npm run build # react-scripts build -> /app/build (plain HTML/CSS/JS)

# ----- stage 2: serve them -----
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html # take ONLY the build output
EXPOSE 80
```

The compilers, the `node_modules` folder, and the source code all stay behind in stage 1 and never reach the final image. The result drops from roughly 1 GB to about 25 MB, boots instantly, and contains no build tooling for an attacker to find.

KEY IDEA A multi-stage build lets you use heavy tools to *produce* your app without shipping those tools *with* your app. `COPY --from=<stage>` is the line that does it.

Where the frontend gets the backend's address

There is one more thing to understand about the frontend, and it explains a manifest you will meet in chapter 15. Look at how the React code calls the API:

```
const apiUrl = process.env.REACT_APP_BACKEND_URL // read at BUILD time, not run time
export function getTasks() { return axios.get(apiUrl); }
```

Create React App bakes any environment variable starting with `REACT_APP_` into the JavaScript bundle when the code is compiled. Because this image runs `npm start` (which compiles on container startup rather than at `docker build` time), setting `REACT_APP_BACKEND_URL` as a container environment variable in the Kubernetes Deployment does work here.

WATCH OUT If you switch to the multi-stage build above, that stops being true. `npm run build` happens during `docker build`, so the variable must be supplied then — via a Docker `ARG` and `--build-arg` — not as a Kubernetes environment variable. This trips up almost everyone the first time they “optimise” a Create React App image. Setting the env var in the Deployment would silently have no effect, and the frontend would call `undefined/api/tasks`.

6. The .dockerignore Files

From the repo: [Application-Code/backend/.dockerignore](#)

Both `frontend/` and `backend/` ship a `.dockerignore`, and they are very nearly identical. This file never appears in the Dockerfile, yet it changes what `COPY . .` actually copies.

What the file does

When you run `docker build`, Docker first packages up the whole folder you pointed it at and sends it to the Docker engine. That package is the **build context**. `.dockerignore` is a list of patterns that are left out of that package — exactly like `.gitignore`, but for builds.

Here is the backend's copy, grouped by purpose:

```
# Logs
logs
*.log
npm-debug.log*

# Runtime data
pids
*.pid
*.seed
*.pid.lock

# Directory used by npm
node_modules
npm-debug.log

# Directory used by yarn
yarn-error.log

# Optional npm cache directory
.npm

# Optional eslint cache
.eslintcache

# Optional REPL history
.node_repl_history

# IDE / editor directories
.vscode/
.idea/
*.swp
*.swo
*.swn
*.bak

# OS generated files
.DS_Store
Thumbs.db
*.lock
```

The frontend's file is the same list plus one extra entry at the end:

```
# Build DIR
build
```

...which keeps any locally compiled React output from being copied in and shadowing what the container produces itself.

Why `node_modules` is the entry that really matters

Of everything in that list, one line does the heavy lifting:

```
node_modules
```

Without it, three things go wrong at once:

- 1 **The build gets slow.** `node_modules` is typically tens of thousands of files and hundreds of megabytes. All of it would be packed up and shipped to the Docker engine before the build even starts.
- 2 **The careful caching from chapter 4 is wasted.** Your local `node_modules` changes whenever you install anything, so `COPY . .` would miss its cache almost every build.
- 3 **It can break the image outright.** `COPY . .` would overwrite the `node_modules` that `npm install` built inside the Linux container with the one from your machine. If you develop on Windows or macOS, any package with compiled native code (Mongoose's BSON parser, for instance) was built for the wrong operating system and will fail at runtime with a cryptic error.

KEY IDEA `.dockerignore` and the `COPY package*.json / RUN npm install` ordering are a **matched pair**. The ordering makes the cache useful; `.dockerignore` keeps stray local files from destroying it. Neither works properly without the other.

TIP Also add `.git` and `.env` to any `.dockerignore` you write yourself. `.git` can be larger than your entire application, and `.env` files contain secrets that have no business being baked into a shared image. Neither is excluded in this project's copy.

7. From Dockerfile to Running Image

The two Dockerfiles are recipes. Here is the short version of how they become images that Kubernetes can actually pull — the bridge between Part II and Part III.

Build

Run `docker build` inside each tier's folder. The `.` at the end is the build context — the folder Docker packages up and sends to the engine:

```
cd Application-Code/backend
docker build -t backend . # -t names ("tags") the resulting image "backend:latest"

cd ../frontend
docker build -t frontend . # same, for the React tier
```

Docker runs the instructions top to bottom, printing one step per line, and ends with the image ID.

Tag for Amazon ECR

An image name is also its address. To push into **Amazon ECR** (AWS's private image registry), the name must start with your registry's hostname, so you re-tag each image:

```
# Log the local Docker daemon into your private ECR registry (token lasts 12 hours)
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin
<your-aws-account-id>.dkr.ecr.us-east-1.amazonaws.com

# Re-tag the local images with their full registry address
docker tag backend:latest
<your-aws-account-id>.dkr.ecr.us-east-1.amazonaws.com/backend:latest
docker tag frontend:latest
<your-aws-account-id>.dkr.ecr.us-east-1.amazonaws.com/frontend:latest
```

`docker tag` does not copy anything — it just adds a second name pointing at the same image.

Push

```
docker push <your-aws-account-id>.dkr.ecr.us-east-1.amazonaws.com/backend:latest
docker push <your-aws-account-id>.dkr.ecr.us-east-1.amazonaws.com/frontend:latest
```

Those two full addresses are what you will see, character for character, in the `image:` field of the backend and frontend Deployments in chapters 13 and 15.

WATCH OUT Because the ECR repositories are **private**, the cluster cannot pull from them without credentials. That is the entire reason both Deployments carry an `imagePullSecrets` block — see chapter 13. Forget it and your pods sit in `ImagePullBackOff` forever.

NOTE Using the `:latest` tag everywhere is convenient but risky: it is a moving target, so you can never tell which build a running pod actually has, and rollbacks become guesswork. Real pipelines tag with something immutable — the Git commit SHA, or a version like `v1.4.2`.

PART III

The Manifests (Kubernetes-Manifests-file/)

All ten YAML files, one chapter at a time: Secret, PV and PVC, three Deployments, three Services, and the ALB Ingress.

8. How to Read These Manifests

From the repo: [Kubernetes-Manifests-file/](#)

Ten YAML files, three folders. Before we open them one by one, here is the map and the two rules that apply to all of them.

The order they must be applied in

Kubernetes objects have dependencies. A pod that mounts a volume cannot start until the volume exists; a pod that reads a Secret cannot start until the Secret exists. So the folders are applied bottom-up, data tier first:

```
kubectl create namespace three-tier # 1. the folder everything lives in
kubectl apply -f Kubernetes-Manifests-file/Database/ # 2. Secret, PV, PVC, Deployment, Service
kubectl apply -f Kubernetes-Manifests-file/Backend/ # 3. the API that talks to Mongo
kubectl apply -f Kubernetes-Manifests-file/Frontend/ # 4. the React app that talks to the API
kubectl apply -f Kubernetes-Manifests-file/ingress.yaml # 5. the public front door
```

NOTE `kubectl apply -f <folder>` reads every `.yaml` file in that folder, but it does **not** descend into sub-folders. Running it once on `Kubernetes-Manifests-file/` would find only `ingress.yaml` and quietly ignore all nine tier manifests. Apply each folder as shown above, or add `-R` to recurse.

TIP Apply order matters less than it seems, because Kubernetes keeps retrying. If you apply the backend before MongoDB is ready, its pods simply crash-loop for a minute and then settle once the database appears. Getting the order right just saves you from a wall of alarming-looking errors.

Rule one — every object is in the `three-tier` namespace

A **namespace** is a folder inside the cluster. Every one of these manifests hard-codes the same one:

```
metadata:
  namespace: three-tier  # hard-coded in all ten files
```

Because the namespace is written *into the files*, it must already exist and it must be spelled exactly `three-tier`. Creating a namespace called something else and applying these manifests will either fail or silently put objects in the wrong place.

It also means every `kubectl` command you run against this app needs `-n three-tier`:

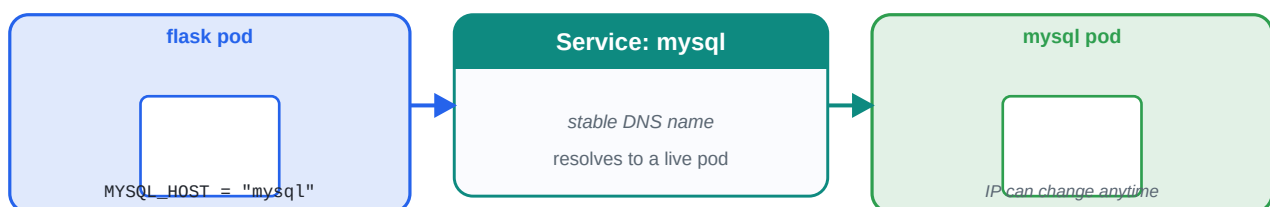
```
kubectl get pods -n three-tier  # without -n, you are looking at the "default" namespace
kubectl get svc -n three-tier
```

TIP Save yourself the typing with `kubectl config set-context --current --namespace=three-tier`. After that, `-n three-tier` is the default for the rest of your session.

Rule two — the labels that wire everything together

Three label values do all the connecting in this project. Learn these three and the rest follows:

Label	Worn by	Searched for by
<code>role: frontend</code>	the React pods	the <code>frontend</code> Deployment and the <code>frontend</code> Service
<code>role: api</code>	the Node.js pods	the <code>api</code> Deployment and the <code>api</code> Service
<code>app: mongodb</code>	the MongoDB pod	the <code>mongodb</code> Deployment and the <code>mongodb-svc</code> Service



Never hard-code a pod's IP. Point to the service NAME and Kubernetes routes to a healthy pod.

Notice the database uses `app:` while the other two use `role:`. The key name is arbitrary — Kubernetes does not care what you call it, only that the **selector matches the label exactly**. The inconsistency here is simply how the original project was written; it is harmless as long as each file is internally consistent.

9. Database — `secrets.yaml`

From the repo: [Kubernetes-Manifests-file/Database/secrets.yaml](#)

We start with the Secret because both the MongoDB Deployment and the backend Deployment read from it.

```
apiVersion: v1
kind: Secret
metadata:
  namespace: three-tier
  name: mongo-sec
type: Opaque
data:
  password: <base64-encoded-password> # echo -n 'your-password' | base64
  username: <base64-encoded-username> # echo -n 'your-username' | base64
```

What a Secret is for

A **Secret** is a small object that holds sensitive values — passwords, tokens, keys — so they do not have to be written inside your Deployment files. The database's Deployment and the API's Deployment both just *refer* to this one object by name, and Kubernetes injects the values into the containers at startup.



A Secret keeps passwords out of your manifests; Kubernetes feeds them into the pod as environment variables.

The practical benefit: you can commit your Deployment YAML to GitHub, and only this one file has to be handled carefully.

Field by field

Field	Meaning
<code>kind:</code> <code>Secret</code>	The object type
<code>name:</code> <code>mongo-sec</code>	The handle other manifests use. Both <code>Database/deployment.yaml</code> and <code>Backend/deployment.yaml</code> reference exactly this string.
<code>type:</code> <code>Opaque</code>	"Arbitrary user-defined key/value data." The general-purpose Secret type. Other types exist for specific formats, such as kubernetes.io/dockerconfigjson for registry logins.
<code>data:</code>	The key/value pairs themselves, with values base64-encoded

Base64 is encoding, not encryption

This is the point beginners most often get wrong, so let us be blunt about it.

Values under `data:` must be base64-encoded. You produce them like this:

```
echo -n 'myuser' | base64 # -n matters: it stops echo adding a newline
# bXl1c2Vy
echo -n 'mypassword' | base64
# bXlwYXNzd29yZA==
```

And anyone with read access to the Secret can undo it just as easily:

```
kubectl get secret mongo-sec -n three-tier -o jsonpath='{.data.password}' | base64 -d
```

WATCH OUT Base64 provides **no security whatsoever**. It exists so that binary data can travel safely inside a YAML file — nothing more. A Secret committed to GitHub is a password published to the internet with one extra step. By default, Kubernetes also stores Secrets **unencrypted** in etcd. Never commit real credentials to a repository.

TIP The `-n` flag in `echo -n` is not optional. Without it, `echo` appends a newline character, that newline gets encoded into your password, and MongoDB rejects the login with an authentication error that gives you no hint at all about the cause. It is a classic hour-long debugging session.

Safer alternatives

For anything beyond a learning project, use one of these instead:

- **Sealed Secrets** (Bitnami) — encrypts the Secret so the encrypted version is safe to commit; only the controller inside your cluster can decrypt it.
- **External Secrets Operator** — leaves the actual value in AWS Secrets Manager or HashiCorp Vault and syncs it into the cluster at runtime.
- **Create it imperatively** and never write it to a file at all:

```
kubectl create secret generic mongo-sec -n three-tier \
  --from-literal=username='myuser' \
  --from-literal=password='mypassword' # kubectl base64-encodes these for you
```

10. Database — pv.yaml and pvc.yaml

From the repo: [Kubernetes-Manifests-file/Database/pv.yaml](#)

These two short files solve one problem: **containers forget everything when they restart**.

A container's filesystem is temporary. Delete the pod and its files go with it. That is fine for a stateless API, and completely unacceptable for a database — you would lose every to-do the moment Kubernetes rescheduled the pod. A **volume** is storage that outlives the pod.



Pods are disposable, so their data must live outside them. The pod claims a PVC, the PVC binds to a PV (real storage) — now MySQL's data survives a pod restart.

The supply side — pv.yaml

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: mongo-pv
  namespace: three-tier
spec:
  capacity:
    storage: 1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data/db
  
```

A **PersistentVolume (PV)** represents an actual piece of storage that exists in the cluster. Think of it as the disk itself.

Field	What it means
<code>capacity.storage: 1Gi</code>	This volume offers 1 gibibyte. Note Gi (1024-based), not G (1000-based).
<code>volumeMode: Filesystem</code>	Present it as a mountable directory, not as a raw block device. This is the usual choice.
<code>accessModes: [ReadWriteOnce]</code>	It can be mounted read-write by one node at a time.
<code>hostPath.path: /data/db</code>	Where the storage physically is: the folder <code>/data/db</code> on whichever worker node the pod lands on.

WATCH OUT `hostPath` means "a folder on the node's own disk". It is the simplest possible volume and the least suitable for production. If the pod is rescheduled onto a **different** node, it finds an empty `/data/db` there — the data has not moved, and to the application it has simply vanished. And if that node is terminated, the data is gone for good.

NOTE A `PersistentVolume` is a **cluster-scoped** object — it does not belong to a namespace. The `namespace: three-tier` line in this file is therefore ignored by Kubernetes. It is harmless, but it is a mistake carried over from the other manifests.

On a real EKS cluster you would delete this file entirely and let the **EBS CSI driver** create a proper network-attached EBS volume on demand, by giving the PVC a `storageClassName: gp3` instead.

The demand side — pvc.yaml

From the repo: [Kubernetes-Manifests-file/Database/pvc.yaml](#)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-volume-claim
  namespace: three-tier
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ""
  resources:
    requests:
      storage: 1Gi
```

A **PersistentVolumeClaim (PVC)** is a *request* for storage. The pod asks for what it needs; Kubernetes finds a PV that satisfies the request and **binds** the two together.

Field	What it means
<code>name: mongo-volume-claim</code>	The name the MongoDB Deployment will reference in its <code>volumes:</code> block
<code>accessModes: [ReadWriteOnce]</code>	Must be compatible with the PV's access mode, or they will never bind
<code>storageClassName: ""</code>	The empty string means "do not dynamically provision anything — bind me to an existing PV" . This is what makes the claim match <code>mongo-pv</code> .
<code>resources.requests.storage: 1Gi</code>	Asks for 1 GiB. The PV offers exactly 1 GiB, so it qualifies.

KEY IDEA PV and PVC are a **supply and demand** pair, like a job advert and an application. The PV says "1 GiB available here"; the PVC says "1 GiB wanted". Kubernetes plays matchmaker and binds them. The pod only ever names the **claim** — it never needs to know where the storage actually is.

TIP `storageClassName: ""` is easy to misread as "not set". It is very deliberate. If you *omit* the field entirely, Kubernetes uses the cluster's default StorageClass and provisions a brand new volume, ignoring `mongo-pv` completely. The explicit empty string switches dynamic provisioning off. You can confirm the match worked with `kubectl get pvc -n three-tier` — you want to see **STATUS: Bound**, not **Pending**.

11. Database — deployment.yaml

From the repo: [Kubernetes-Manifests-file/Database/deployment.yaml](#)

Now MongoDB itself. This is the first full Deployment, so we will take it slowly — the structure repeats for the other two tiers.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: three-tier
```

```

name: mongodb
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mon
          image: mongo:4.4.6
          command:
            - "numactl"
            - "--interleave=all"
            - "mongod"
            - "--wiredTigerCacheSizeGB"
            - "0.1"
            - "--bind_ip"
            - "0.0.0.0"
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongo-sec
                  key: username
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-sec
                  key: password
          volumeMounts:
            - name: mongo-volume
              mountPath: /data/db
      volumes:
        - name: mongo-volume
          persistentVolumeClaim:
            claimName: mongo-volume-claim

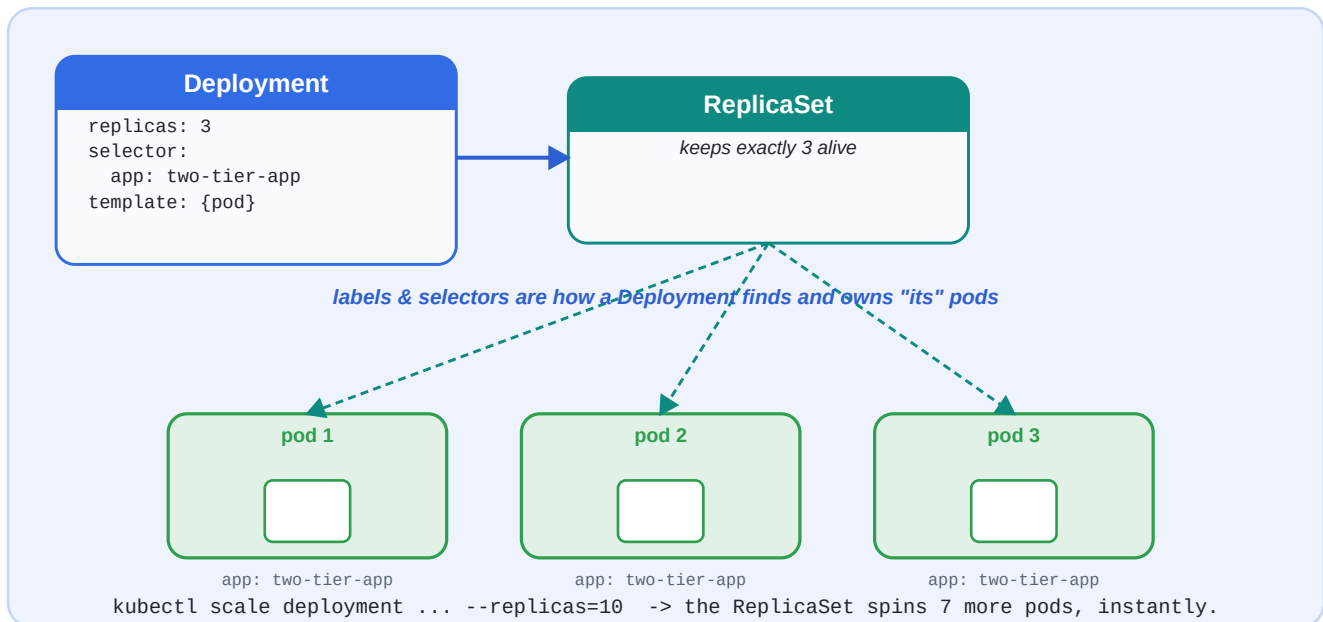
```

The two spec blocks — the thing that confuses everyone

Look carefully and you will notice **spec**: **appears twice**, and **metadata**: appears twice. That is not a typo, and understanding it makes every Deployment readable:

```
spec: # <- the DEPLOYMENT's spec: how many pods, which pods are mine
  replicas: 1
  selector: ...
  template: # <- a pod BLUEPRINT starts here
    metadata: # <- the POD's metadata (its labels)
      labels: ...
    spec: # <- the POD's spec: containers, volumes
      containers: ...
```

The outer `spec` is about **managing pods**. Everything under `template:` is a complete pod definition — a stamp the Deployment uses to create each pod. When you edit anything under `template`, the Deployment notices and rolls out replacement pods.



replicas: 1 — and why it must stay 1 here

```
replicas: 1 # exactly one MongoDB pod
```

For the API you would happily run several copies. For this database you must not, for two reasons:

- 1 The PV uses `ReadWriteOnce` — only one node can mount it at a time.
- 2 Two MongoDB processes writing to the same data directory will corrupt it.

Scaling a database is not a matter of raising `replicas`; it needs a replica set, which in Kubernetes means a `StatefulSet` rather than a `Deployment`.

NOTE A `Deployment` is really designed for *stateless* workloads — interchangeable pods with no identity. Databases want stable names, stable storage, and ordered startup, which is exactly what a `StatefulSet` provides. Running MongoDB as a `Deployment` with one replica works fine for a demo, and it is a fair thing to raise if an interviewer asks how you would harden this project.

selector **and** `template.metadata.labels` **must match**

```

selector:
  matchLabels:
    app: mongodb # "the pods I manage wear this sticker"
template:
  metadata:
    labels:
      app: mongodb # "...and here I am putting that sticker on them"

```

The Deployment identifies its own pods by this label. The same label is also what `mongodb-svc` will search for in chapter 12.

WATCH OUT These two must match, and `selector` is **immutable** after creation. If you later decide to rename the label, you cannot edit it in place — you must delete the Deployment and recreate it. Kubernetes will reject the update with a validation error.

The container — name and image

```

- name: mon # the container's name inside the pod
  image: mongo:4.4.6 # the official MongoDB image, pinned to version 4.4.6

```

`name: mon` is just an identifier. You use it when a pod has several containers, for example `kubectl logs <pod> -c mon`.

`image: mongo:4.4.6` is pulled from Docker Hub. Unlike the frontend and backend images, this one is public, so no pull secret is needed. Note that it is **pinned to an exact version** — which is good practice, and a nice contrast with the `:latest` tags used for the app tiers.

command: — overriding the image's own startup command

```

command:
  - "numactl"
  - "--interleave=all"
  - "mongod"
  - "--wiredTigerCacheSizeGB"
  - "0.1"
  - "--bind_ip"
  - "0.0.0.0"

```

This is the Kubernetes equivalent of overriding `CMD` from chapter 4 — it replaces whatever the `mongo` image would normally run. Read as one shell line it is:

```
numactl --interleave=all mongod --wiredTigerCacheSizeGB 0.1 --bind_ip 0.0.0.0
```

Each piece:

Part	What it does
<code>numactl --interleave=all</code>	Spreads memory allocation evenly across CPU memory banks. MongoDB's own documentation recommends it; on a small single-CPU node it changes very little.
<code>mongod</code>	The MongoDB server process itself
<code>--wiredTigerCacheSizeGB 0.1</code>	Caps MongoDB's in-memory cache at 100 MB. Without it, MongoDB grabs roughly half the node's RAM and can get the pod evicted on a small worker node.

Part	What it does
<code>--bind_ip 0.0.0.0</code>	Listen on all network interfaces, not just localhost. Essential — otherwise the pod would only accept connections from inside itself and the API could never reach it.

TIP `--bind_ip 0.0.0.0` is the line that makes the database reachable at all. If you ever see a containerised database that works fine with `kubectl exec` but refuses every connection from other pods, a missing bind-address setting is the first thing to check.

ports — documentation, mostly

```
ports:
- containerPort: 27017 # MongoDB's standard port
```

This declares that the container listens on 27017. It is worth knowing that `containerPort` does **not** open or restrict anything — the process is reachable on that port whether or not you declare it. The field exists so that humans and tools reading the manifest know what the container does.

env with `secretKeyRef` — credentials without hard-coding

```
env:
- name: MONGO_INITDB_ROOT_USERNAME # the variable name inside the container
  valueFrom: # don't take a literal value...
    secretKeyRef: # ...take it from a Secret
      name: mongo-sec # which Secret (from chapter 9)
      key: username # which key inside it
```

`MONGO_INITDB_ROOT_USERNAME` and `MONGO_INITDB_ROOT_PASSWORD` are variables the official `mongo` image looks for. When the data directory is empty on first start, it creates an administrator account with those credentials.

The `valueFrom` / `secretKeyRef` pattern is the important part: the Deployment file names the Secret and the key, never the value. Kubernetes looks it up, base64-decodes it, and sets the variable inside the container.

NOTE `MONGO_INITDB_*` only takes effect on a **completely empty** data directory. Because `/data/db` is a persistent volume here, changing the password in the Secret later will not change the database user — the directory is no longer empty, so the image skips initialisation entirely.

WATCH OUT If the Secret does not exist, or the key name is misspelled, the pod stays in `CreateContainerConfigError` and never starts. `kubectl describe pod` tells you exactly which key it could not find — always read it before guessing.

volumeMounts and volumes — the storage handshake

These two blocks always come as a pair, and they live at different levels:

```
volumeMounts: # inside the CONTAINER: where to attach it
- name: mongo-volume
  mountPath: /data/db # MongoDB's data directory
volumes: # inside the POD: what the volume actually is
- name: mongo-volume # the same name links the two
  persistentVolumeClaim:
    claimName: mongo-volume-claim # the PVC from chapter 10
```

Read it as two sentences:

- 1 **volumes** (pod level): "This pod has a volume called `mongo-volume`; its contents come from the claim `mongo-volume-claim`."
- 2 **volumeMounts** (container level): "Attach the volume called `mongo-volume` at the path `/data/db` inside this container."

The shared string `mongo-volume` is what joins them, and it is purely local to this pod — it has nothing to do with the PV's or PVC's names.

So the full chain is:

```
mongod writes to /data/db -> mongo-volume -> mongo-volume-claim (PVC) -> mongo-pv (PV) -> /data/db on the node
```

KEY IDEA Storage in Kubernetes is deliberately indirect, and each hop has a purpose. The container knows only a mount path. The pod knows only a claim name. The claim knows only a size and an access mode. Only the PV knows where the bytes really are — which is what lets you swap `hostPath` for an EBS volume without touching the Deployment at all.

12. Database — `service.yaml`

From the repo: [Kubernetes-Manifests-file/Database/service.yaml](#)

```
apiVersion: v1
kind: Service
metadata:
  namespace: three-tier
  name: mongodb-svc
spec:
  selector:
    app: mongodb
  ports:
    - name: mongodb-svc
      protocol: TCP
      port: 27017
      targetPort: 27017
```

Why a Service is needed at all

Pods are disposable. Every time one restarts it gets a **new IP address**. If the backend hard-coded the MongoDB pod's IP, it would break the first time the database pod was rescheduled.

A **Service** solves this with a fixed name and a fixed virtual IP. It keeps a live list of the pods matching its selector and load-balances traffic across them. The name never changes, no matter how many times the pods behind it do.

Three types of Service

ClusterIP	NodePort	LoadBalancer
default	30000-32767	cloud only
Reachable ONLY inside the cluster.	Opens a port on every node's IP.	Cloud provisions a real load balancer + URL.
Use it for the database tier (pod-to-pod).	Reach the app at NodeIP:30007.	Best on EKS / managed Kubernetes.

Field by field

Field	What it means
<code>name: mongodb-svc</code>	Becomes a DNS name inside the cluster. This exact string appears in the backend's connection string.
<code>selector.app: mongodb</code>	Sends traffic to every pod labelled <code>app: mongodb</code> — the pod from chapter 11.
<code>port: 27017</code>	The port the Service listens on
<code>targetPort: 27017</code>	The port on the pod to forward to
<code>protocol: TCP</code>	TCP (the default; <code>UDP</code> and <code>SCTP</code> also exist)
<code>ports[].name</code>	A label for this port. Optional for a single-port Service, required when there is more than one.

NOTE `port` and `targetPort` are allowed to differ — a Service can listen on 80 and forward to a container on 8080. Here both are 27017, which is common and slightly obscures the distinction. `port` is the front door; `targetPort` is the door on the container.

No type: means ClusterIP

This Service does not specify a `type`, so it gets the default: `ClusterIP`. That means it is reachable **only from inside the cluster** — which is exactly right for a database. Nothing on the internet can reach MongoDB, and that is by design.

The DNS name this creates

Kubernetes' internal DNS gives every Service a name. This one can be reached as:

Name	Works from
<code>mongodb-svc</code>	any pod in the <code>three-tier</code> namespace
<code>mongodb-svc.three-tier</code>	any pod in any namespace
<code>mongodb-svc.three-tier.svc.cluster.local</code>	anywhere — the full canonical name

The backend uses the short form, which is why this Service and the API pods must be in the same namespace.

TIP To prove a Service is wired up correctly, run `kubectl get endpoints mongodb-svc -n three-tier`. If you see a pod IP and port, the selector matched. If you see `<none>`, the selector matched nothing — check your labels before you debug anything else.

13. Backend — deployment.yaml

From the repo: [Kubernetes-Manifests-file/Backend/deployment.yaml](https://github.com/kubernetes-manifests-file/Backend/deployment.yaml)

The API tier. This is the richest manifest in the project — it introduces rolling updates, private image pulls, and health probes.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: three-tier
  labels:
    role: api
    env: demo
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 25%
  selector:
    matchLabels:
      role: api
  template:
    metadata:
      labels:
        role: api
    spec:
      imagePullSecrets:
        - name: ecr-registry-secret
      containers:
        - name: api
          image: <your-aws-account-id>.dkr.ecr.us-east-1.amazonaws.com/backend:latest
          imagePullPolicy: Always
          env:
            - name: MONGO_CONN_STR
              value: mongodb://mongodb-svc:27017/todo?directConnection=true
            - name: MONGO_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongo-sec
                  key: username
            - name: MONGO_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-sec
                  key: password
```

```

ports:
- containerPort: 3500
livenessProbe:
  httpGet:
    path: /ok
    port: 3500
  initialDelaySeconds: 2
  periodSeconds: 5
readinessProbe:
  httpGet:
    path: /ok
    port: 3500
  initialDelaySeconds: 5
  periodSeconds: 5
  successThreshold: 1

```

replicas: 2 — why the API can scale but the database cannot

```
replicas: 2 # two identical API pods
```

The API is **stateless**: it holds nothing in memory between requests, and every pod talks to the same MongoDB. So any pod can serve any request, and running two gives you both extra capacity and survivability — if one pod dies, the Service keeps sending traffic to the other while Kubernetes starts a replacement.

That is precisely what the database *cannot* do, for the reasons in chapter 11. Stateless tiers scale horizontally; stateful ones need more care.

strategy — how new versions replace old ones

```

strategy:
  type: RollingUpdate # replace pods gradually, never all at once
  rollingUpdate:
    maxSurge: 1 # may run 1 pod ABOVE the desired count during the rollout
    maxUnavailable: 25% # may have 25% of pods unavailable during the rollout

```

When you change the image, the Deployment does not kill both pods and start two new ones — that would mean downtime. **RollingUpdate** replaces them one at a time, and the two knobs bound how far it may stray from the target:

Setting	Meaning with replicas: 2
maxSurge: 1	May temporarily run up to 3 pods (2 + 1), so a new pod can start before an old one stops
maxUnavailable: 25%	Of 2 pods, 25% rounds down to 0 — so it must always keep both old pods serving until replacements are ready

The effect: a new pod starts, passes its readiness probe, joins the Service, and only then is an old pod removed. Users see no interruption.

NOTE Both values accept either a number or a percentage. **maxUnavailable: 0** combined with **maxSurge: 1** gives the safest possible rollout (never fewer healthy pods than you asked for), at the cost of needing capacity for one extra pod. The alternative strategy, **type: Recreate**, kills everything first and then starts the new version — brief downtime, but sometimes necessary when two versions cannot run at once.

imagePullSecrets — permission to pull a private image

```
spec:
  imagePullSecrets:
    - name: ecr-registry-secret # the registry login created with kubectl
```

The image lives in a **private** ECR repository. Without credentials, the node's attempt to pull it is rejected and the pod sits in [ImagePullBackOff](#).

This field points at a Secret of type [kubernetes.io/dockerconfigjson](#) — a stored registry login. It is not in the repo, because it contains a live token; you create it by hand before deploying:

```
kubectl create secret docker-registry ecr-registry-secret \
  --docker-server=<your-aws-account-id>.dkr.ecr.us-east-1.amazonaws.com \
  --docker-username=AWS \
  --docker-password=$(aws ecr get-login-password --region us-east-1) \
  --namespace three-tier
```

WATCH OUT The ECR password produced by [get-login-password](#) is a token that **expires after 12 hours**. Once it does, *new* pods can no longer pull the image — although pods already running keep going, which makes the failure appear at a confusing, unrelated moment. Re-create the Secret, or use an IAM role attached to the node group so no token is needed at all.

NOTE Notice where this field sits: at the **pod** level ([template.spec](#)), alongside [containers:](#), not inside a container. It applies to every container in the pod.

image and imagePullPolicy

```
image: <your-aws-account-id>.dkr.ecr.us-east-1.amazonaws.com/backend:latest
imagePullPolicy: Always # re-pull on every pod start, ignore the node's cache
```

The image address is exactly what you pushed in chapter 7 — registry host, then repository, then tag. [<your-aws-account-id>](#) is a placeholder you must replace with your 12-digit AWS account number.

[imagePullPolicy: Always](#) tells the node to check the registry for a newer image every time a pod starts, instead of reusing whatever it has cached. That is necessary here precisely *because* the tag is [:latest](#): the tag stays the same while the image behind it changes, so the cache would otherwise serve stale code forever.

TIP If you tag images immutably ([backend:v1.4.2](#)), you can use [imagePullPolicy: IfNotPresent](#) and get faster pod starts, because a given tag always means the same image. [Always](#) is the workaround for mutable tags, not a best practice in itself.

The three environment variables

```
- name: MONGO_CONN_STR
  value: mongodb://mongodb-svc:27017/todo?directConnection=true
```

This is the line where the middle tier finds the data tier. Breaking the URL apart:

Piece	Meaning
mongodb://	The MongoDB protocol
mongodb-svc	The Service name from chapter 12 — resolved by cluster DNS, no IP address anywhere
:27017	The Service's port

Piece	Meaning
/todo	The database name to use
?directConnection=true	Connect to this one server directly instead of trying to discover a replica set

This single string is the whole point of Services. The backend never knows or cares which pod, on which node, with which IP, is currently running MongoDB.

The other two variables come from the Secret, using the same `secretKeyRef` pattern explained in chapter 11:

```
- name: MONGO_USERNAME
  valueFrom:
    secretKeyRef:
      name: mongo-sec
      key: username
```

NOTE A quirk worth knowing: looking at the app's `db.js`, these credentials are only actually used if a `USE_DB_AUTH` variable is set — and this Deployment never sets it. So the API connects **without authentication** in this configuration. The variables are supplied and then ignored. Real deployments must set `USE_DB_AUTH` and connect authenticated.

livenessProbe and readinessProbe — two different questions

This pair is the most valuable thing in the file, and beginners mix them up constantly.

```
livenessProbe: # "Are you ALIVE? If not, restart the container."
  httpGet:
    path: /ok
    port: 3500
  initialDelaySeconds: 2 # wait 2s after container start before the first check
  periodSeconds: 5 # then check every 5s
readinessProbe: # "Are you READY for traffic? If not, take you out of the Service."
  httpGet:
    path: /ok
    port: 3500
  initialDelaySeconds: 5
  periodSeconds: 5
  successThreshold: 1 # one successful check is enough to be marked ready
```

Both call `GET /ok` on port 3500 — an endpoint the app defines specifically for this:

```
app.get('/ok', (req, res) => { res.status(200).send('ok') }) // a health-check endpoint
```

The difference is entirely in **what Kubernetes does when the check fails**:

Probe	Question	On failure
<code>livenessProbe</code>	"Is this process healthy?"	Kill and restart the container
<code>readinessProbe</code>	"Can this pod serve requests right now?"	Remove the pod from the Service's endpoints — no restart

Readiness is the gentler one. A pod that is still starting up, or briefly overloaded, is taken out of rotation and put back when it recovers — no restart, no lost requests. Liveness is the blunt instrument for a process that has hung and will never recover on its own.

WATCH OUT Getting these backwards causes real outages. If you use a liveness probe where you meant readiness, a slow-starting pod gets killed, restarts, is still slow, and gets killed again — a crash loop caused entirely by the probe. A good rule: **always** define a readiness probe; add a liveness probe only when you have a genuine hang scenario, and give it a generous `initialDelaySeconds`.

TIP Readiness probes are also what makes the rolling update from earlier in this chapter safe. A new pod joins the Service only once its readiness probe passes — so traffic never reaches a pod that is still booting.

14. Backend — service.yaml

From the repo: [Kubernetes-Manifests-file/Backend/service.yaml](#)

```
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: three-tier
spec:
  ports:
    - port: 3500
      protocol: TCP
  type: ClusterIP
  selector:
    role: api
```

Structurally this is the same as the database Service from chapter 12, with three details worth noting.

type: ClusterIP is stated explicitly. The database Service left it out and got the same default. Both are correct; writing it out is clearer for a reader.

There is no targetPort. When `targetPort` is omitted, Kubernetes assumes it is the same as `port` — so traffic to the Service on 3500 goes to the pods on 3500. That is right here, but it is worth being aware that this is a default filling in a blank rather than a deliberate statement.

It load-balances over both API pods. The selector `role: api` matches the two replicas from chapter 13, so this one Service spreads requests across them.

NOTE The Service is named `api` while the Deployment is also named `api` — Kubernetes is happy with that, because names only need to be unique *within a kind*. The pods, though, are named after the Deployment (`api-7d9f8b6c5-x2k4p`), so `kubectl get all -n three-tier` will show `service/api` and `deployment.apps/api` side by side.

TIP `ClusterIP` here is a real security decision. The API is not directly reachable from the internet — the only way in is through the Ingress in chapter 17, which routes only the `/api` path. Exposing the backend as a `LoadBalancer` "to test it" would quietly put your database-backed API on the public internet.

15. Frontend — deployment.yaml

From the repo: [Kubernetes-Manifests-file/Frontend/deployment.yaml](#)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: three-tier
  labels:
    role: frontend
    env: demo
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 25%
  selector:
    matchLabels:
      role: frontend
  template:
    metadata:
      labels:
        role: frontend
    spec:
      imagePullSecrets:
        - name: ecr-registry-secret
      containers:
        - name: frontend
          image: <your-aws-account-id>.dkr.ecr.us-east-1.amazonaws.com/frontend:latest
          imagePullPolicy: Always
          env:
            - name: REACT_APP_BACKEND_URL
              value: "http://<your-domain>/api/tasks"
          ports:
            - containerPort: 3000

```

Nearly everything here was covered in chapter 13 — same rolling-update strategy, same [imagePullSecrets](#), same [imagePullPolicy: Always](#). Three things differ.

[replicas: 1 — and no probes](#)

The frontend runs a single pod and defines no liveness or readiness probe. Both are simplifications for a demo: with one replica there is nothing to load-balance across, and with no readiness probe the pod is considered ready the moment its container starts — before the React dev server has finished compiling. In practice that means a few seconds of failed requests after every rollout.

Adding a readiness probe on `/` and a second replica would be the first two improvements to make.

[REACT_APP_BACKEND_URL — the address the browser will call](#)

```

env:
  - name: REACT_APP_BACKEND_URL
    value: "http://<your-domain>/api/tasks"

```

This is the most commonly misunderstood line in the whole project, and it is worth slowing down for.

You might expect this to be <http://api:3500/api/tasks> — the in-cluster Service name, like the backend uses for MongoDB. **It must not be.** The reason is *who does the calling*.

The React code does not run on the server. It is downloaded and executed **in the user's browser**, out on the public internet. That browser has never heard of your cluster's internal DNS, so <http://api:3500> means nothing to it. The URL must therefore be one the browser can reach: the public address of your Ingress.

Call	Made by	Address used	Why
Frontend to backend	The user's browser	<a href="http://<your-domain>/api/tasks">http://<your-domain>/api/tasks — public	The browser is outside the cluster
Backend to MongoDB	The API pod	mongodb://mongodb-svc:27017 — internal	The pod is inside the cluster

KEY IDEA "Frontend talks to backend" is misleading. The *frontend pod* never talks to the backend at all — it only hands JavaScript to a browser, and the **browser** calls the backend. That is why frontend URLs are always public addresses while backend-to-database URLs are internal Service names. Once this clicks, a whole class of three-tier networking confusion disappears.

[<your-domain>](#) is a placeholder. Replace it with the hostname you put in the Ingress in chapter 17 — and the two must match exactly, because the ALB routes by `Host` header.

TIP If the page loads but the to-do list stays empty, open your browser's developer console. A failed call to <http://<your-domain>/api/tasks> (with the placeholder still in it) or a CORS error tells you immediately that this value is wrong. Because the variable is compiled into the JavaScript, changing it means restarting the pod: `kubectl rollout restart deployment/frontend -n three-tier`.

`containerPort: 3000`

Create React App's dev server listens on 3000, so that is what this declares — and what the frontend Service forwards to in chapter 16.

16. Frontend — service.yaml

From the repo: [Kubernetes-Manifests-file/Frontend/service.yaml](#)

```
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: three-tier
spec:
  ports:
    - port: 3000
      protocol: TCP
  type: ClusterIP
  selector:
    role: frontend
```

The shortest manifest in the project, and by now entirely predictable: a `ClusterIP` Service on port 3000, selecting pods labelled `role: frontend`.

The interesting question is why the *frontend* — the one tier users actually visit — is still `ClusterIP` and not `LoadBalancer`.

The answer is that the Ingress in the next chapter is the single public entry point for the whole app. It needs to route two paths (`/` and `/api`) to two different Services through **one** address and **one** load balancer. Making this Service a `LoadBalancer` would create a second AWS load balancer, on a different address, bypassing the routing entirely — and AWS would bill you for it.

KEY IDEA In an Ingress-based architecture, **all** application Services stay `ClusterIP`. The Ingress controller is the only thing that talks to the outside world. One public address, one load balancer, one bill, one place to attach TLS certificates.

17. The Ingress — `ingress.yaml`

From the repo: [Kubernetes-Manifests-file/ingress.yaml](https://kubernetes.io/docs/concepts/services-networking/ingress-nginx/#ingress-yaml)

The last manifest, and the one that makes the app reachable from a browser.

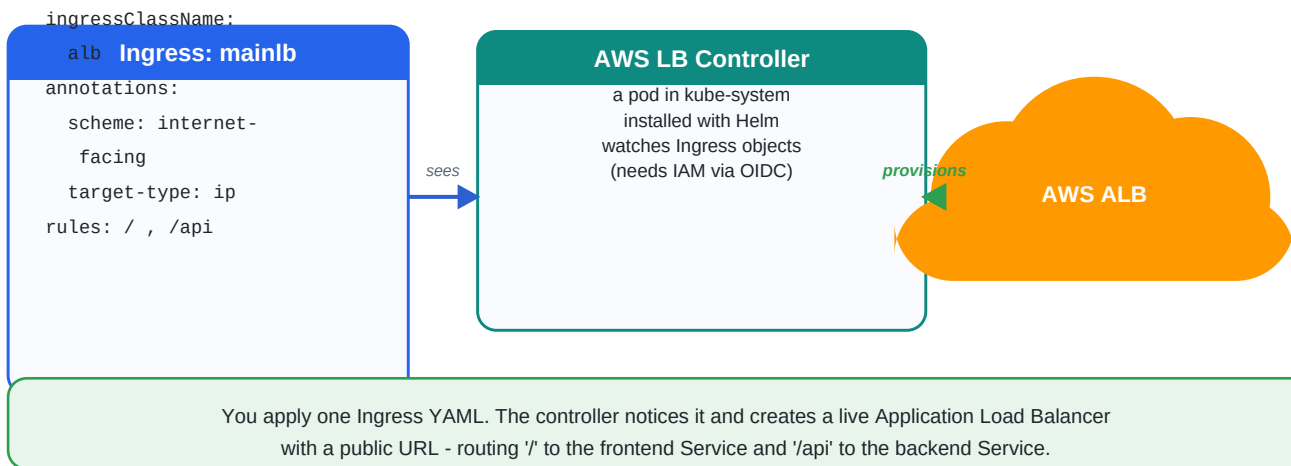
```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mainlb
  namespace: three-tier
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}]'
spec:
  ingressClassName: alb
  rules:
    - host: <your-domain>
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 3500
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 3000
```

What an Ingress is — and what it is not

An **Ingress** is a set of HTTP routing rules. On its own it does **nothing at all** — it is just a description sitting in the cluster.

Something has to read that description and build real infrastructure from it. That something is an **Ingress controller**. On EKS it is the **AWS Load Balancer Controller**, which watches for Ingress objects and creates a genuine **Application Load Balancer** in your AWS account to match.

How an Ingress becomes a real AWS load balancer



WATCH OUT If the AWS Load Balancer Controller is not installed, applying this file appears to succeed — and then nothing happens. `kubectl get ingress -n three-tier` shows an empty `ADDRESS` column forever. The controller is installed separately with Helm, together with an IAM service account that gives it permission to create load balancers. Its absence is the single most common reason this project "doesn't work" at the last step.

ingressClassName: alb — choosing which controller responds

```
ingressClassName: alb # "AWS Load Balancer Controller, this one is yours"
```

A cluster can run several Ingress controllers — NGINX, Traefik, ALB. This field says which one should act on this object. `alb` is the class registered by the AWS Load Balancer Controller.

The annotations — configuring the ALB itself

Annotations are free-form key/value notes on an object. The Ingress spec has no field for "make this internet-facing", so AWS-specific settings are passed as annotations that only its controller understands:

Annotation	What it does
<code>alb.ingress.kubernetes.io/scheme: internet-facing</code>	Give the ALB a public IP. The alternative, <code>internal</code> , would be reachable only from inside your VPC.
<code>alb.ingress.kubernetes.io/target-type: ip</code>	Send traffic straight to pod IPs , rather than to node ports. Fewer hops, and it requires the VPC CNI (standard on EKS).
<code>alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}]'</code>	The ALB listens on port 80 for plain HTTP.

WATCH OUT `HTTP: 80` means **no encryption**. Every request, including anything a user types, crosses the internet in plain text. For anything real, request a certificate in AWS Certificate Manager and switch to `listen-ports: '[{"HTTPS": 443}]'` plus a `certificate-arn` annotation, with an HTTP-to-HTTPS redirect.

NOTE `target-type: ip` is also what makes this compatible with the `readinessProbe` from chapter 13. With IP targets, the ALB registers individual pods, so a pod that is not ready is not sent traffic.

The routing rules

rules:

```
- host: <your-domain> # only requests with this Host header are matched
```

The rules apply only to requests whose `Host` header matches. This is the same `<your-domain>` placeholder from the frontend Deployment in chapter 15, and **the two must be identical** — the frontend tells the browser to call that hostname, and the ALB only recognises requests arriving with it.

Then two paths, in this order:

```
- path: /api # checked first
  pathType: Prefix
  backend:
    service:
      name: api # -> the api Service (chapter 14)
      port:
        number: 3500
- path: / # the catch-all
  pathType: Prefix
  backend:
    service:
      name: frontend # -> the frontend Service (chapter 16)
      port:
        number: 3000
```

`pathType: Prefix` means "match this path and everything under it". So:

Request	Matches	Goes to
GET /	/	frontend Service, port 3000
GET /static/js/main.js	/	frontend Service, port 3000
GET /api/tasks	/api	api Service, port 3500
POST /api/tasks	/api	api Service, port 3500

KEY IDEA Both rules would match `/api/tasks` — `/` is a prefix of everything. The ALB resolves this the way routers generally do: **the most specific (longest) matching prefix wins**. `/api` is longer than `/`, so API calls go to the API. If they were evaluated purely top to bottom instead, putting `/` first would swallow every request, and nothing would ever reach the backend. Listing the specific path first is good practice regardless.

Finding the address

Once the controller has built the ALB, its hostname appears here:

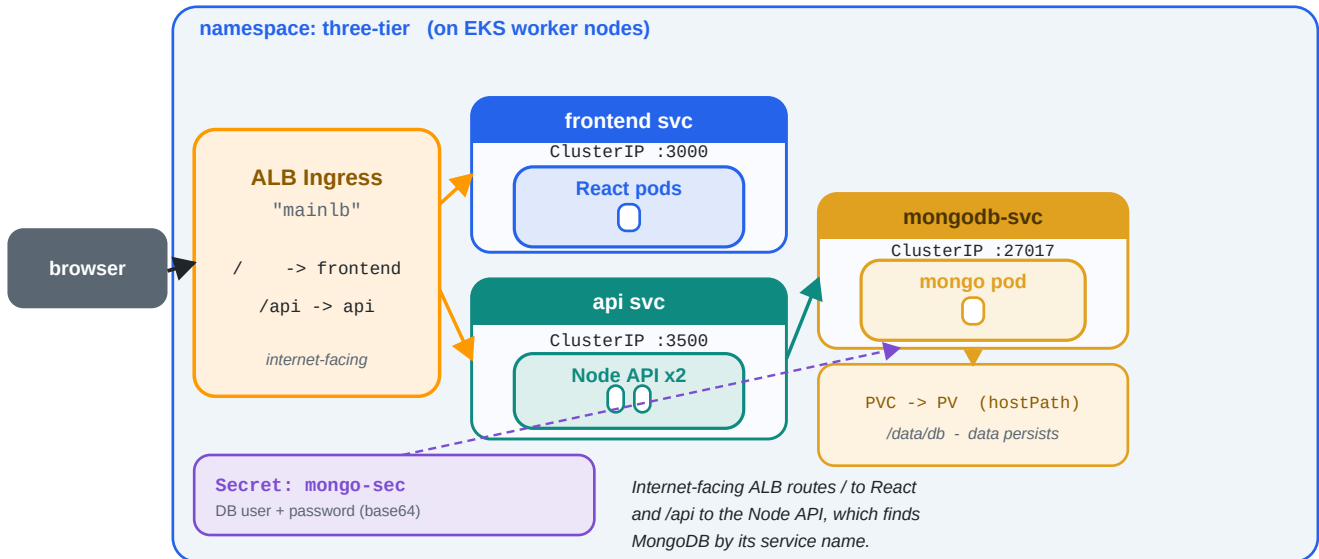
```
kubectl get ingress -n three-tier
# NAME CLASS HOSTS ADDRESS PORTS
# mainlb alb <your-domain> k8s-threetie-mainlb-xxxx.us-east-1.elb.amazonaws.com 80
```

Point your domain's DNS at that address with a CNAME (or a Route 53 alias record), and the app is live. It usually takes two to three minutes for the ALB to be provisioned and pass health checks.

18. Putting It All Together

You have now read all ten manifests. Here is the whole journey of a single click, using only the objects you have seen.

The three-tier app on Amazon EKS



A user adds a to-do

- 1 The user opens <http://<your-domain>/>. DNS points at the **ALB** built by `ingress.yaml`.
- 2 The ALB matches path `/` and forwards to the **frontend Service** on port 3000, which forwards to the frontend pod (matched by `role: frontend`).
- 3 The pod serves the React application. The browser downloads it and starts running it.
- 4 The user types a task and clicks Add. The **browser** calls <http://<your-domain>/api/tasks> — the value of `REACT_APP_BACKEND_URL`.
- 5 That request hits the same ALB, which now matches the longer prefix `/api` and forwards to the **api Service** on port 3500.
- 6 The Service load-balances to one of the two API pods (matched by `role: api`), but only to pods whose **readiness probe** is passing.
- 7 The Express app handles `POST /api/tasks` and asks Mongoose to save. It connects using `MONGO_CONN_STR`, whose hostname `mongodb-svc` is resolved by cluster DNS.
- 8 The **mongodb-svc Service** forwards to the MongoDB pod (matched by `app: mongod`).
- 9 `mongod` writes to `/data/db`, which is the **PersistentVolumeClaim** `mongo-volume-claim`, bound to the **PersistentVolume** `mongo-pv`. The to-do is now on disk and survives a pod restart.
- 10 The response travels back the same way, and the new task appears on the page.

Which file did what

Step	File that made it possible
Public address and path routing	<code>ingress.yaml</code>
Stable name for each tier	the three <code>service.yaml</code> files
Pods actually running	the three <code>deployment.yaml</code> files
Credentials without hard-coding	<code>Database/secrets.yaml</code>
Data surviving a restart	<code>Database/pv.yaml</code> + <code>Database/pvc.yaml</code>
The images the pods run	the two <code>Dockerfiles</code> from Part II

Verify it yourself

```
kubectl get pods -n three-tier # all pods should be Running and READY 1/1
kubectl get svc -n three-tier # three ClusterIP services
kubectl get pvc -n three-tier # mongo-volume-claim should say "Bound"
kubectl get endpoints -n three-tier # every service should list at least one pod IP
kubectl get ingress -n three-tier # ADDRESS should hold the ALB hostname
kubectl logs deployment/api -n three-tier # look for "Connected to database."
```

That last line — [Connected to database.](#) — is printed by the backend's `db.js` and is the single best confirmation that all three tiers found each other.

APPENDICES

Appendices

A full field reference, the placeholders you must replace, common errors decoded, interview questions, a glossary, and every link.

19. Field Reference Cheat Sheet

Every field used anywhere in this project, in one place.

Dockerfile instructions

Instruction	Runs at	Used in this project for
FROM node:14	build	The base image — Linux plus the Node 14 runtime
WORKDIR /usr/src/app	build	Setting the working folder for all later instructions
COPY package*.json ./	build	Copying only the dependency list, so its layer caches well
RUN npm install	build	Downloading dependencies into the image
COPY . .	build	Copying the application source
CMD ["node", "index.js"]	run	Starting the API (backend)
CMD ["npm", "start"]	run	Starting the React dev server (frontend)

Fields common to every manifest

Field	Purpose
apiVersion	Which API group and version the object belongs to
kind	The object type
metadata.name	The object's name, unique within its kind and namespace
metadata.namespace	The namespace it lives in — always <code>three-tier</code> here
metadata.labels	Key/value stickers used by selectors
spec	The desired state

Deployment fields

Field	Purpose
<code>spec.replicas</code>	How many identical pods to keep running
<code>spec.selector.matchLabels</code>	Which pods this Deployment owns. Immutable after creation.
<code>spec.strategy.type</code>	RollingUpdate (gradual) or Recreate (all at once)
<code>spec.strategy.rollingUpdate.maxSurge</code>	How many pods above the target may exist mid-rollout
<code>spec.strategy.rollingUpdate.maxUnavailable</code>	How many pods may be missing mid-rollout
<code>spec.template.metadata.labels</code>	The labels stamped onto each pod — must match the selector
<code>spec.template.spec.containers[].name</code>	The container's name inside the pod
<code>spec.template.spec.containers[].image</code>	The image to pull
<code>spec.template.spec.containers[].imagePullPolicy</code>	Always , IfNotPresent , or Never
<code>spec.template.spec.containers[].command</code>	Overrides the image's CMD
<code>spec.template.spec.containers[].ports.containerPort</code>	Documents the port the app listens on
<code>spec.template.spec.containers[].env[].value</code>	A literal environment variable
<code>spec.template.spec.containers[].env[].valueFrom.secretKeyRef</code>	An environment variable read from a Secret
<code>spec.template.spec.containers[].volumeMounts</code>	Where to attach a volume inside the container
<code>spec.template.spec.containers[].livenessProbe</code>	Health check — failure restarts the container
<code>spec.template.spec.containers[].readinessProbe</code>	Traffic check — failure removes the pod from Services
<code>spec.template.spec.imagePullSecrets</code>	Registry credentials, at pod level
<code>spec.template.spec.volumes</code>	What the pod's volumes actually are

Service fields

Field	Purpose
<code>spec.type</code>	ClusterIP (internal, the default), NodePort , or LoadBalancer
<code>spec.selector</code>	Which pods to send traffic to
<code>spec.ports[].port</code>	The port the Service listens on
<code>spec.ports[].targetPort</code>	The port on the pod. Defaults to port if omitted.
<code>spec.ports[].protocol</code>	TCP (default), UDP , or SCTP

Storage and Secret fields

Field	Purpose
PV spec.capacity.storage	How much storage this volume offers
PV spec.accessModes	ReadWriteOnce , ReadOnlyMany , or ReadWriteMany
PV spec.hostPath.path	A folder on the node — demo-only storage
PVC spec.resources.requests.storage	How much storage the pod wants
PVC spec.storageClassName: ""	Bind to an existing PV; do not dynamically provision
Secret type: Opaque	Arbitrary key/value data
Secret data	Values, base64-encoded

Ingress fields

Field	Purpose
spec.ingressClassName	Which controller should act on this Ingress (alb)
spec.rules[].host	Only requests with this Host header match
spec.rules[].http.paths[].path	The URL path to match
spec.rules[].http.paths[].pathType	Prefix , Exact , or ImplementationSpecific
spec.rules[].http.paths[].backend.service	The Service and port to forward to
annotations: alb.ingress.kubernetes.io/scheme	internet-facing or internal
annotations: alb.ingress.kubernetes.io/target-type	ip (to pods) or instance (to node ports)
annotations: alb.ingress.kubernetes.io/listen-ports	Which ports and protocols the ALB listens on

20. Placeholders You Must Replace

The repository ships with deliberate placeholders so that no real account details are published. Nothing will deploy until you replace all four.

Placeholder	Appears in	Replace with
<your-aws-account-id>	Backend/deployment.yaml, Frontend/deployment.yaml	Your 12-digit AWS account number, e.g. 123456789012
<your-domain>	Frontend/deployment.yaml, ingress.yaml	The hostname you will point at the ALB. Must be identical in both files.
<base64-encoded-username>	Database/secrets.yaml	Output of <code>`echo -n 'youruser' \ </code>
<base64-encoded-password>	Database/secrets.yaml	Output of <code>`echo -n 'yourpass' \ </code>

Also check the AWS **region**. The manifests hard-code [us-east-1](#) inside the ECR image URLs. If you build and push in a different region, those URLs must change to match.

Find every remaining placeholder before you deploy:

```
grep -rn "<your-" Kubernetes-Manifests-file/ # lists every file and line still to fill in
```

21. Common Errors and What They Mean

The failures you are most likely to hit, and which chapter explains the cause.

What you see	What it means	Fix
<code>ImagePullBackOff</code>	The node cannot pull the image	Check <code>imagePullSecrets</code> and that the ECR token has not expired (chapter 13)
<code>ErrImagePull</code> with <code>not found</code>	The image address is wrong	Check the account ID, region and repository name in <code>image</code> :
<code>CreateContainerConfigError</code>	A referenced Secret or key does not exist	Apply <code>secrets.yaml</code> first; check the key names match (chapter 9)
Pod <code>Pending</code> , PVC <code>Pending</code>	No PV matched the claim	Apply <code>pv.yaml</code> ; check sizes and access modes agree (chapter 10)
<code>CrashLoopBackOff</code> on the API	It cannot reach MongoDB, or a probe is failing	<code>kubectl logs</code> ; check <code>MONGO_CONN_STR</code> and give Mongo time to start
Service exists but nothing responds	Selector does not match any pod labels	<code>kubectl get endpoints -n three-tier --<none></code> confirms it (chapter 8)
Ingress <code>ADDRESS</code> stays empty	No AWS Load Balancer Controller is installed	Install it with Helm plus its IAM service account (chapter 17)
Page loads, to-do list empty	<code>REACT_APP_BACKEND_URL</code> is wrong	Check the browser console; the value must match the Ingress host (chapter 15)
<code>no matches for kind "Ingress"</code>	Wrong <code>apiVersion</code>	Use <code>networking.k8s.io/v1</code>
Objects apply but are not found	They went into the wrong namespace	Every command needs <code>-n three-tier</code> (chapter 8)

The three commands that diagnose almost everything:

```
kubectl describe pod <pod-name> -n three-tier # the Events list at the bottom names the real cause
```

```
kubectl logs <pod-name> -n three-tier # what the application itself printed
```

```
kubectl get endpoints -n three-tier # proves whether Services found their pods
```

22. Interview Questions and Answers

Q. Why does the Dockerfile copy `package*.json` and run `npm install` before copying the rest of the source?

A. Because of Docker's layer cache. Dependencies change rarely; source code changes constantly. By installing dependencies in their own layer first, an ordinary code edit only invalidates the final `COPY . .` layer, and the slow `npm install` is skipped. If you copied everything first, every code change would force a full re-install.

Q. What is the difference between `RUN` and `CMD`?

A. `RUN` executes during `docker build` and its result is baked into the image as a layer. `CMD` executes when a container starts, and sets the default command. `RUN npm install` puts libraries in the image; `CMD ["node", "index.js"]` starts the server.

Q. What does `.dockerignore` do, and why is excluding `node_modules` so important?

A. It excludes files from the build context, so they are never sent to the Docker engine or copied by `COPY`.
.. Excluding `node_modules` keeps the build fast, protects the layer cache, and — most importantly — stops your host machine's modules from overwriting the ones installed inside the Linux container, which would break any package containing compiled native code.

Q. What is a multi-stage build and why would you use one here?

A. It is a Dockerfile with more than one `FROM`, where a later stage copies artefacts from an earlier one using `COPY --from=`. For the React frontend you would build the static files in a Node stage, then copy only the `build/` folder into an `nginx:alpine` image. The compilers and `node_modules` never reach the final image, taking it from about 1 GB to about 25 MB.

Q. Why are all three Services `ClusterIP` when users need to reach the app?

A. Because the Ingress is the single public entry point. The AWS Load Balancer Controller creates one ALB that routes `/` to the frontend and `/api` to the backend. Making a Service `LoadBalancer` would create a second, separate load balancer — extra cost, a different address, and it would bypass the routing.

Q. Why does the frontend use a public URL for the backend while the backend uses an internal Service name for MongoDB?

A. Because the React code runs in the user's browser, outside the cluster, where internal DNS names mean nothing — so it needs the public Ingress address. The backend runs as a pod inside the cluster, so it can and should use the internal Service name `mongodb-svc`.

Q. What is the difference between a liveness probe and a readiness probe?

A. They ask different questions and have different consequences. A failing liveness probe means the process is unhealthy, so Kubernetes restarts the container. A failing readiness probe means the pod cannot serve traffic right now, so Kubernetes removes it from its Services' endpoints without restarting it. Readiness protects users from a pod that is still starting; liveness recovers a process that has hung.

Q. What do `maxSurge` and `maxUnavailable` control?

A. How far a rolling update may deviate from the desired replica count. `maxSurge: 1` permits one pod above the target so a replacement can start before an old pod stops; `maxUnavailable: 25%` caps how many pods may be missing at once. With two replicas, 25% rounds down to zero, so both old pods keep serving until new ones are ready.

Q. How do a `PersistentVolume` and a `PersistentVolumeClaim` relate?

A. The PV is the supply — an actual piece of storage with a size and access mode. The PVC is the demand — a pod's request for storage. Kubernetes binds a claim to a volume that satisfies it. The pod references only the claim, which is what lets you swap the underlying storage without editing the Deployment.

Q. What does `storageClassName: ""` mean in the PVC?

A. It explicitly disables dynamic provisioning, telling Kubernetes to bind to an existing PV instead of creating a new volume. Omitting the field entirely would instead use the cluster's default StorageClass and provision fresh storage, ignoring `mongo-pv`.

Q. Kubernetes Secrets are only base64-encoded. Is that secure?

A. No. Base64 is encoding for safe transport, not encryption, and anyone who can read the Secret can decode it in one command. By default Secrets are also stored unencrypted in etcd. For production, enable encryption at rest and use Sealed Secrets or an external secret store such as AWS Secrets Manager.

Q. Why does this Deployment need `imagePullSecrets`?

A. The images live in a private Amazon ECR repository, so the node must authenticate before it can pull them. `imagePullSecrets` points at a `docker-registry` Secret holding that login. Without it, pods fail with `ImagePullBackOff`. Note that ECR tokens expire after 12 hours, so an IAM role on the node group is the more robust approach.

Q. Why `imagePullPolicy: Always`?

A. Because the images are tagged `:latest`, which is mutable — the same tag points at different images over time. `Always` forces the node to check the registry rather than reuse a cached copy. With immutable tags such as a Git SHA, `IfNotPresent` would be better and faster.

Q. Why is MongoDB a Deployment with one replica rather than three?

A. Because its PersistentVolume is `ReadWriteOnce` — mountable by one node at a time — and two `mongod` processes writing the same data directory would corrupt it. Scaling a database needs a replica set, which in Kubernetes means a `StatefulSet` with per-pod storage, not a higher `replicas` count.

Q. Why is `hostPath` unsuitable for production?

A. It stores data in a folder on one specific node's local disk. If the pod is rescheduled to another node, it finds an empty directory and the data appears to have vanished; if the node is terminated, the data is lost. On EKS you would use the EBS CSI driver to provision network-attached volumes instead.

Q. The Ingress defines both `/api` and `/`. Why doesn't `/` swallow every request?

A. Because prefix routing resolves ties by choosing the longest matching prefix. `/api/tasks` matches both rules, but `/api` is more specific, so it wins. Listing the specific path before the catch-all is good practice in any case.

Q. What does `alb.ingress.kubernetes.io/target-type: ip` change?

A. It makes the ALB register pod IP addresses directly as targets rather than node ports. That removes a hop through `kube-proxy` and lets the ALB respect pod readiness — traffic goes only to pods that are ready.

Q. An Ingress was applied but no load balancer appeared. Why?

A. An Ingress is only a description; an Ingress controller must act on it. On EKS that is the AWS Load Balancer Controller, installed separately with Helm and an IAM service account. Without it, the object exists and `ADDRESS` stays empty forever.

Q. How do a Deployment and a Service find their pods?

A. Entirely through labels. The Deployment's `selector.matchLabels` identifies the pods it owns; a Service's `selector` identifies the pods it sends traffic to. Nothing is wired by IP. If a selector does not match, the Service has no endpoints and traffic silently goes nowhere.

Q. `kubectl apply -f Kubernetes-Manifests-file/` seemed to work but only the Ingress was created. Why?

A. `apply -f <folder>` does not descend into sub-folders. It found `ingress.yaml` at the top level and ignored `Database/`, `Backend/` and `Frontend/`. Apply each folder individually, or add `-R` to recurse.

23. Glossary

Term	Meaning
ALB	AWS Application Load Balancer — the layer-7 load balancer the Ingress controller creates
Annotation	Free-form key/value metadata on an object, used to pass controller-specific settings
Base image	The image a Dockerfile builds on top of, named in <code>FROM</code>
Base64	An encoding that represents binary data as text. Not encryption.
Build context	The folder sent to the Docker engine when you run <code>docker build</code> , filtered by <code>.dockerignore</code>
ClusterIP	The default Service type — reachable only from inside the cluster
Container	A running instance of an image
ECR	Amazon Elastic Container Registry — AWS's private image registry
EKS	Amazon Elastic Kubernetes Service — managed Kubernetes on AWS

Term	Meaning
Endpoint	The pod IP and port a Service currently forwards to
Image	A read-only package containing an application and everything it needs to run
Ingress	A set of HTTP routing rules; needs a controller to become real infrastructure
Ingress controller	The component that watches Ingress objects and configures a real load balancer
Label	A key/value sticker on an object, matched by selectors
Layer	A filesystem snapshot created by one Dockerfile instruction; the unit of Docker's cache
Liveness probe	A health check whose failure restarts the container
Manifest	A YAML file describing a Kubernetes object
Multi-stage build	A Dockerfile with several FROM stages, so build tools stay out of the final image
Namespace	A folder inside the cluster that groups related objects
Pod	The smallest deployable unit — one or more containers sharing a network address
PV	PersistentVolume — a piece of storage that exists independently of any pod
PVC	PersistentVolumeClaim — a request for storage, bound to a PV
Readiness probe	A traffic check whose failure removes the pod from Service endpoints
Reconciliation	The loop by which Kubernetes continuously makes reality match your manifests
Registry	A server that stores and serves images, such as Docker Hub or ECR
Replica	One of the identical pods a Deployment maintains
Rolling update	Replacing pods gradually so the app stays available during a deployment
Secret	An object holding sensitive key/value data, injected into containers
Selector	A label query that determines which objects a controller or Service acts on
Service	A stable name and virtual IP in front of a changing set of pods
StatefulSet	The workload controller for stateful apps — stable names, stable storage, ordered startup
Tag	The part of an image name after the colon, identifying a version
Three-tier architecture	Splitting an app into presentation, application and data tiers
WORKDIR	The Dockerfile instruction that sets the working directory inside the image

24. Links and Repository

The source code

All the code explained in this guide is in the author's GitHub repository. Every Dockerfile and manifest quoted here is copied from it verbatim.

From the repo: github.com/nitin1094/three-tier-app-eks

What	Where
The repository	github.com/nitin1094/three-tier-app-eks
Backend Dockerfile	Application-Code/backend/Dockerfile
Frontend Dockerfile	Application-Code/frontend/Dockerfile
All Kubernetes manifests	Kubernetes-Manifests-file/
Full deployment walkthrough	README.md

Clone it and read along:

```
git clone https://github.com/nitin1094/three-tier-app-eks.git
cd three-tier-app-eks
```

Official documentation

Topic	Link
Dockerfile reference	docs.docker.com/reference/dockerfile
Dockerfile best practices	docs.docker.com/build/building/best-practices
Multi-stage builds	docs.docker.com/build/building/multi-stage
Kubernetes Deployments	kubernetes.io/docs/concepts/workloads/controllers/deployment
Kubernetes Services	kubernetes.io/docs/concepts/services-networking/service
Kubernetes Ingress	kubernetes.io/docs/concepts/services-networking/ingress
Secrets	kubernetes.io/docs/concepts/configuration/secret
Persistent Volumes	kubernetes.io/docs/concepts/storage/persistent-volumes
Liveness and readiness probes	kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes

Topic	Link
AWS Load Balancer Controller	kubernetes-sigs.github.io/aws-load-balancer-controller
Amazon ECR	docs.aws.amazon.com/AmazonECR

Where to go next

Now that the files make sense, the natural next steps are:

- 1 Deploy it** — follow the step-by-step guide in the repository's [README.md](#).
- 2 Harden it** — convert the frontend Dockerfile to a multi-stage nginx build, move MongoDB to a [StatefulSet](#) with an EBS-backed volume, add HTTPS to the Ingress, and replace the plain Secret with Sealed Secrets.
- 3 Automate it** — put the [docker build](#) and [kubectl apply](#) steps behind a CI/CD pipeline, or manage the manifests declaratively with Argo CD.